

Оглавление

Экзаменационный билет №1	2
Экзаменационный билет №2	3
Экзаменационный билет №3	4
Экзаменационный билет №4	5
Экзаменационный билет №5	7
Экзаменационный билет №6	8
Экзаменационный билет №7	9
Экзаменационный билет №8	10
Экзаменационный билет №9	11
Экзаменационный билет №10	12
Экзаменационный билет №11	13
Экзаменационный билет №12	14
Экзаменационный билет №13	15
Экзаменационный билет №14	16
Экзаменационный билет №15	17
Экзаменационный билет №16	18
Экзаменационный билет №17	19
Экзаменационный билет №18	20
Экзаменационный билет №19	21
Экзаменационный билет №20	22
Экзаменационный билет №21	23
Экзаменационный билет №22	24
Экзаменационный билет №23	25
Экзаменационный билет №24	26
Экзаменационный билет №25	27
Экзаменационный билет №26	28
Экзаменационный билет №27	29
Экзаменационный билет №28	30
Экзаменационный билет №29	31
Экзаменационный билет №30	32

Экзаменационный билет №1

1. Молодой программист Вася реализовал класс «Человек», в котором необходимо реализовать поле «Возраст»:

```
class Person {  
public:  
    int age;  
};
```

Однако старший программист Вова не принял его работу, сказав, что данный класс недоработан. В чём состоит недоработка Васи? Покажите Васе доработанный код, который устраивает Вову.

```
class Person {  
public:  
    void setAge(int A) {age = A;}  
    int getAge() {return age;}  
  
private:  
    int age;  
};
```

Недоработка класса «Человек», согласно замечанию Вовы, заключается в том, что поле age не должно быть напрямую доступным извне класса, так как это нарушает принцип инкапсуляции.

2. Для чего используются дружественные функции и классы? Какие существуют правила описания дружественных функций? Каким образом можно перегрузить унарные операции? Привести фрагмент кода на C++ с описанием класса и определением перегруженной унарной функции-операции.

Дружественные функции и классы в C++ предоставляют доступ другим функциям или классам к закрытой части (приватной или защищенной) другого класса.

1. Объявляются внутри класса со спецификатором friend, но определяются вне.
2. Параметры передаются только по ссылке, значению, адресу
3. Не является методом класса
4. Не использует указатель this
5. Дружественный класс должен быть другом другому классу

```
friend ostream& operator <<(ostream& out, const Person& P);
```

Экзаменационный билет №2

1. Ученик по имени Арсений полгода назад написал функцию, однако в то время он все функции и переменные именовал одной буквой, и теперь смотря на этот код, не может вспомнить, что же он написал.

Расскажите Арсению, что за функцию он реализовал:

```
void a(const char* b) {  
    fstream c(b, ios::out | ios::trunc);  
    if (!c) throw -1;  
    int d, e, f;  
    cin >> d;  
    for (e = 0; e < d; e++) {  
        cin >> f;  
        c << f << "\n";  
    }  
    c.close();  
}
```

Функция для записи чисел в файл. **b** – имя файла.

2. Для чего используются дружественные функции и классы? Чем отличается перегрузка префиксных и постфиксных унарных операций? Каким образом можно перегрузить операции ввода-вывода? Привести фрагмент кода на C++ с описанием класса и определением перегруженных операций ввода-вывода.

Дружественные функции и классы в C++ предоставляют доступ другим функциям или классам к закрытой части (приватной или защищенной) другого класса.

Префиксный выполняется до возвращения значения

Постфиксный возвращает значение и выполняет операцию

Для перегрузки операторов ввода-вывода необходимо перегрузить `operator<<` `operator>>`

```
friend ostream& operator <<(ostream& out, const Person& P);  
friend istream& operator >>(istream& in, Person& P);
```

Экзаменационный билет №3

1. Программист Павел реализовал класс «Время», в котором есть поле, хранящее время в миллисекундах. В основной программе то и время ему необходимо выводит на экран время не в миллисекундах, а в стандартном виде: Минуты:Секунды:Миллисекунды.

```
class Time
private:
    int _milliseconds;
public:
    Time(int milliseconds) {
        _milliseconds = milliseconds;
    }
    ...//другие методы
};
```

Павел не желает по многу раз переписывать один и тот же код с пересчётом минут и секунд. Так же Павел требует, чтобы вывод в основной программе работал через следующие коды:

```
Time myOwnTime(6542960);
cout << myOwnTime;
ofstream writer("MyOwnTime.txt");
writer << myOwnTime;
```

Дополните функционал класса, так, чтобы выполнить все требования Павла.

```
int getMilliseconds() { return _milliseconds % 1000;}
int getMinutes(){ return _milliseconds / (60 * 1000);}
int getSeconds() {return (_milliseconds / 1000) % 60;}
friend ostream& operator<<(std::ostream& os, const Time& t);
```

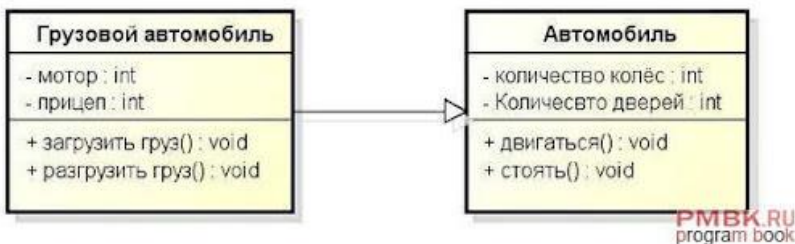
2. Для чего используется принцип наследования? Каким образом наследуются компоненты классов, описанных различными спецификаторами доступа? Наследуются ли конструкторы? Деструкторы? Каким образом описывается производный класс? Привести пример UML-диаграммы базового класса и производного ему.

Наследование – описание нового класса (дочернего) на основе существующего (родительского).

Наследование по умолчанию private, но может быть public, protected.

Конструкторы и деструкторы не наследуются.

```
class грузCar : public Car {
public:
    void грузCar();
};
```



Экзаменационный билет №4

1. Виктор работает на предприятии, которое проектирует ракеты. Разрабатываются множество типов ракет различного назначения:

Грузовые ракеты;

Ракеты-модули для МКС;

Баллистические ракеты с ядерными боеголовками;

Все вышеописанные ракеты имеют несколько общих параметров:

1. Масса ракеты;

2. Дальность полёта;

Также все ракеты имеют одинаковые функции:

1. Запуск ракеты;

2. Вывод сообщения о текущем местоположении;

Виктор понимает, что у ракет много общих параметров, соответственно, код в классах всех ракет будет неизбежно повторяться, а при необходимости изменить одну из функций заставит Виктора переписывать их несколько раз, а на подходе новые типы ракет.

Опишите заголовочные файлы классов ракет так, чтобы Виктору не пришлось переписывать код при необходимости правок. К каждому типу ракет добавьте свой параметр.

```
class Rocket {
protected:
    double mass;
    double range;

public:
    Rocket(double m, double r);
    virtual void launch();
    virtual void location();
};
```

```
class CargoRocket : public Rocket {
private:
    double груз; // Грузоподъемность

public:
    CargoRocket(double m, double r, double g);
    void launch() override;
    void location() override;
};
```

```
class MKSRocket : public Rocket {
private:
    string modul; // Грузоподъемность

public:
    MKSRocket(double m, double r, string mod);
    void launch() override;
    void location() override;
};
```

```
class BallRocket : public Rocket {
private:
    int num; // Грузоподъемность

public:
    BallRocket(double m, double r, int n);
    void launch() override;
    void location() override;
};
```

2. Для чего используется принцип наследования? Что представляют собой виртуальные функции и механизм позднего связывания? Могут ли быть виртуальными конструкторы? Деструкторы? В чём заключается принцип подстановки? Привести фрагмент кода на C++, показывающий работу принципа подстановки.

Наследование – описание нового класса (дочернего) на основе существующего (родительского). Виртуальные функции позволяют определить функцию в родительском классе и переопределить в дочернем.

Полиморфизм (механизм позднего связывания) — вызывает методы дочернего класса через родительский.

Конструкторы и деструкторы могут быть виртуальными, но они не наследуются, поэтому смысла нет.

Принцип подстановки — объекты одного типа могут быть заменены объектами другого типа.

Экзаменационный билет №5

1. Представлен класс «Список»:

```
struct ListItem {  
    int Data;  
    ListItem* Next;  
};  
class UserList {  
private:  
    ListItem* firstItem;  
    int _length = 0;  
};
```

Необходимо реализовать метод, который добавляет один элемент в конец списка

```
void push(int Data) {  
    node* newNode = new node;  
    newNode->data = Data;  
    newNode->next = top;  
    if (front == nullptr) {  
        front = newNode;  
    }  
    else {  
        back->next = newNode;  
    }  
    back = newNode;  
}
```

2. Какой метод называется чисто виртуальным? Чем он отличается от виртуального? Какой класс называется абстрактным? Для чего он используется? Привести пример кода на C++ с описанием абстрактного класса.

Чисто виртуальный метод содержит признак = 0, вместо тела: `virtual void def(int) = 0`. Не реализуется в базовом классе, реализуется в производных классах. Класс, содержащий чисто виртуальный метод – абстрактный.

```
class Shape {  
public:  
    virtual void draw() const = 0; // Чисто виртуальный метод  
};  
  
class Circle : public Shape {  
public:  
    void draw() const override {  
        cout << "Drawing a circle" << endl;  
    }  
};
```

Экзаменационный билет №6

1. Никита давным-давно написал программу, которая управляется событиями. Структуру события он задал следующим образом:

```
Struct TEvent {  
    int what; // определяет тип события  
    union { // определение типа, хранящегося в TEvent  
        int command;  
        struct {  
            int message; // передача сообщения  
            int additional; // передача дополнительных данных о событии  
        };  
    };  
};
```

Но он забыл, какие переменные за что отвечают. Напомните Никите предназначение каждого поля структуры события.

2. Что такое полиморфные функции? Чем полиморфизм отличается от принципа подстановки? В каких случаях используется механизм позднего связывания? Привести фрагмент кода на C++ с примером использования полиморфной функции.

Полиморфизм - способность объекта использовать методы аналогичным способом, независимо от типа этого объекта.

Принцип подстановки - объекты определенного класса А могут заменяться их подклассами В без изменения корректности программы.

МПС используется, когда вызываемый метод определяется во время выполнения программы на основе типа объекта, к которому обращается метод.

```
void drawShape(const Shape& shape) {  
    shape.draw();  
}
```


Экзаменационный билет №7

1. Студент Алексей реализовал функцию вычисления наибольшего корня квадратного уравнения, однако в процессе тестирования он замечает, что в некоторых случаях программа вместо того, чтобы вывести ответ внезапно перестаёт выполняться. Код его функции:

```
double QuadricEquation(double A, double B, double C) {  
    double discriminant = B * B - 4 * A * C;  
    double root = (-B + sqrt(discriminant)) / (2 * A);  
    return root;  
}
```

Алексей не знает, как обрабатывать исключительные ситуации. Помогите ему реализовать корректную обработку исключительных ситуаций в функции и в основной программе.

```
double QuadricEquation(double A, double B, double C) {  
    double discriminant = B * B - 4 * A * C;  
  
    if (discriminant < 0) {  
        throw err("Квадратное уравнение не имеет действительных корней");  
    }  
  
    if (A == 0) {  
        throw err("Коэффициент A не может быть равен нулю");  
    }  
  
    double root = (-B + sqrt(discriminant)) / (2 * A);  
    return root;  
}
```

2. Что такое абстрактный тип данных? Что такое контейнер? Какие виды доступа к элементам контейнера существуют? Привести примеры.

Абстрактный тип данных (АТД) - тип данных, определяемый только через операции (класс, структура, список...)

Контейнер - набор однотипных элементов (массив, список, стек, очередь, двусторонняя очередь...)

1. Последовательный (перемещение через итератор)
2. Прямой (по индексу)
3. Ассоциативный (по содержимому)

Экзаменационный билет №8

1. Студент Алексей реализовал функцию вычисления наибольшего корня квадратного уравнения, однако в процессе тестирования он замечает, что в некоторых случаях программа вместо того, чтобы вывести ответ внезапно перестаёт выполняться. Код его функции:

```
double QuadricEquation(double A, double B, double C) {  
    double discriminant = B * B - 4 * A * C;  
    double root = (-B + sqrt(discriminant)) / (2 * A);  
    return root;  
}
```

Алексей не знает, как обрабатывать исключительные ситуации. Помогите ему реализовать корректную обработку исключительных ситуаций в функции и в основной программе.

```
double QuadricEquation(double A, double B, double C) {  
    double discriminant = B * B - 4 * A * C;  
  
    if (discriminant < 0) {  
        throw err("Квадратное уравнение не имеет действительных корней");  
    }  
  
    if (A == 0) {  
        throw err("Коэффициент A не может быть равен нулю");  
    }  
  
    double root = (-B + sqrt(discriminant)) / (2 * A);  
    return root;  
}
```

2. Что такое контейнер? Какие группы операций выделяют в контейнерах? Что такое итератор и каким образом его можно реализовать? Привести примеры операций итератора.

Контейнер - набор однотипных элементов (массив, список, стек, очередь, двусторонняя очередь...)

- Последовательный (перемещение через итератор)
- Прямой (по индексу)
- Ассоциативный (по содержимому)

Итератор – объект для перебора элементов в контейнере.

```
vector<int>::iterator it;  
for (it = vec.begin(); it != vec.end(); it++) {  
    cout << *it << " ";  
}
```

Экзаменационный билет №9

1. Программист Александр моделирует животный мир, его интересуют несколько видов домашних животных:

1. Кошка;
2. Собака;
3. Корова;

Помимо того, что все они умеют ходить, они ещё могут издавать звуки, однако они это делают по-разному.

Александр необходимо не только реализовать их общий класс и произвести наследование, но и реализовать методы издавания их звуков таким образом, чтобы:

1. Названия методов каждого животного совпадало;
2. Животные издавали разные звуки;

Помогите Александру реализовать данную задачу. В качестве звуков животных используйте печать на экран через `std::cout`.

```
class Animal {
public:
    void walk();
    virtual void makeSound();
};
```

```
class Cat : public Animal {
public:
    void makeSound() override {
        cout << "Мяу-мяу" << endl;
    }
};
```

2. Что такое шаблон? Перечислите основные свойства параметризованных классов. Приведите пример явного инстанцирования шаблона.

Шаблон – обобщение класса или функции, работающий с разными типами данных во избежание дублирования функции/класса.

Свойства:

- Определяется 1 раз
- Определение шаблона параметризуется
- Не могут быть виртуальными
- Могут быть производными от шаблона или от класса
- Могут являться базовыми

Инстанцирование – использование шаблонного класса с конкретным типом данных

```
MyTemplate<int> intTemplate;
intTemplate.printValue(42);
```

Экзаменационный билет №10

1. Ученик по имени Фёдор изучает программирование на C++. Недавно он познакомился с шаблонами, и для закрепления материала решил написать свой параметризованный класс, расписав два файла:

1. Заголовочный файл класса со всеми объявлениями полей и методов;
2. Файл .cpp с реализацией методов;

Заголовочный файл Calculator.h

```
#pragma once
template <class T>
class Calculator {
public:
    T Add(T a, T b);
    T Multiply(T a, T b);
};
```

Файл с реализацией Calculator.cpp

```
#include "Calculator.h"
template <class T>
T Calculator::Add(T a, T b) { return a + b; }
template <class T>
T Calculator::Multiply(T a, T b) { return a * b; }
```

Однако Фёдору не удаётся скомпилировать полученный код. Фёдор в недоумении, синтаксических ошибок нет. В чём заключается ошибка в его коде? Расскажите, как её исправить.

```
template <class T>
T Calculator<T>::Add(T a, T b) {return a + b;}

template <class T>
T Calculator<T>::Multiply(T a, T b) {return a * b;}
```

2. Что такое шаблон? Перечислите основные свойства шаблонов функций. Приведите пример неявного инстанцирования шаблона.

Шаблон – обобщение класса или функции, работающий с разными типами данных во избежание дублирования функции/класса.

Свойства:

- Определяется 1 раз
- Определение шаблона параметризуется
- Не могут быть виртуальными
- Могут быть производными от шаблона или от класса
- Могут являться базовыми

Инстанцирование – использование шаблонного класса с конкретным типом данных

```
Array<int> intArray(5);
```

Экзаменационный билет №11

1. Программист Валера решил реализовать программу для решения матриц, но застрял на этапе реализации функции добавления элемента в матрицу. Помогите Валере реализовать функцию добавления элемента в матрицу, используя уже реализованные классы.

```
struct MatrixItem{
public:
    int value;
};
class Matrix{
    vector<vector<MatrixItem *>> matrix;
public:
    bool add(size_t x, size_t y, MatrixItem *item);
};
```

```
bool add(size_t x, size_t y, MatrixItem* item) {
    if (x >= matrix.size() || y >= matrix[0].size()) {
        return false; // Индекс выходит за пределы матрицы
    }
    matrix[x][y] = item; // Добавляем элемент
    return true;
}
```

2. Что такое класс-группа? Какой вид иерархии даёт группа? Привести пример описания класса-группы Список (List) и пример метода для просмотра её элементов.

Класс-группа - это объект, в который включены другие объекты (элементы группы). Эти элементы могут быть группой. Например: огород состоящий из растений, системы полива и плана выращивания.

```
class List {
private:
    T* next = nullptr;
    int data;
    size = 0;

public:
    List(int D);
    void show() {
        for (int i = 0; i < size; i++) {
            cout << elements[i] << " ";
        }
        cout << endl;
    }
};
```

Экзаменационный билет №12

1. Диме понадобилось реализовать программу, управляемую событиями. Для каждого события должно вычисляться некоторое значение X типа `float` по формуле ``value_a * value_b - (value_b / value_a)``. Чтобы не создавать внешнюю функцию и не захламлять ужасный код, Дима решил создать класс-наследник класса события. Помогите Диме добавить наследнику функцию для вычисления значения X .

```
class Event {
protected:
    int event_id;
    int value_a;
    double value_b;
    void *something;
};
```

```
template <class T>
float calculateX() const override {
    return value_a * value_b - (value_b / value_a);
}
```

2. Что такое событие? Для чего они используются? Какие характеристики должно иметь событие-сообщение? Привести пример структуры, описывающей событие.

События - пакеты информации, которыми обмениваются объекты и которые создаются ОО средой в ответ на действия пользователя. С точки зрения C++ - объект, у которого отдельные поля характеризуют свойства передаваемой информации.

- код класса сообщения, отличающий сообщения одного класса от другого.
- адрес объекта, которому предназначено сообщение
- информационное поле

```
struct Event {
    int what;
    union {
        int command;
        struct {
            int message, parametrCommand;
        };
    }
};
```

Экзаменационный билет №13

1. Программист Слава 2 года назад написал программу, которая находит некоторое значение, а какое - не помнит. Разберитесь в этом хаосе и расскажите Славе, какое же значение находит его программа, а заодно и переименуйте переменные (и функцию), чтоб такой путаницы больше не возникло.

```
double V(double b, double c)
{
    return (4. / 3) * 3.14 * b * b * b * c;
}
```

Функция вычисляет объем шара.

```
double calculateVolume(double radius, double coefficient)
{
    return (4.0 / 3.0) * 3.14 * radius * radius * radius * coefficient;
}
```

2. Что представляет собой исключение на C++? На какие части исключения позволяют разделить вычислительный процесс? Каковы достоинства такого подхода?

Исключение - непредвиденное аварийное событие. В C++ - объект, который система должна генерировать при возникновении исключительной ситуации.

- Обнаружение аварийной ситуации (неизвестно как обрабатывать)
- Обработка аварийной ситуации (неизвестно где возникла)
- Достоинства:
- Удобно в программе из нескольких модулей.
- Не требуется возвращать значение в вызывающую функцию

Экзаменационный билет №14

1. Студент Алексей реализовал программу для вычисления куба числа v , однако при введении некоторых чисел программа выдает неверный ответ. Исправьте программу, чтобы она правильно работала. Код его функции:

```
int pow(int v, int c)
{
    for (int i = 0; i < c; i++)
        v *= v * v;
    return v;
}

int calculateCube(int v, int c)
{
    int result = 1;

    for (int i = 0; i < c; i++)
    {
        result *= v;
    }

    return result;
}
```

2. Что представляет собой исключение на C++? Какой оператор используется для генерации исключительной ситуации? Что такое контролируемый блок? Секция-ловушка? Для чего они нужны? Как можно создать собственную иерархию исключений?

Исключение - непредвиденное аварийное событие. В C++ - объект, который система должна генерировать при возникновении исключительной ситуации.

Оператор: throw <выражение>, выражение: константа, переменная.

Для проверки возникновения исключения используется контролируемый блок try {}, с которым связана одна или несколько секций-ловушек catch {}. Необходимо для перехвата исключения и его обработки.

Экзаменационный билет №15

1. Программист Илья реализовывал класс-наследник, с выводом в файл, но что-то пошло не так, и теперь код выводит данные только класса наследника. Помогите Илье исправить класс-наследник. Чтобы упростить Вам задачу, он убрал свой код из функции `operator<<`, так что теперь вы можете написать эту функцию с чистого листа.

```
class Abs{
```

```
private:
```

```
    int value_a;
```

```
    double value_b;
```

```
public:
```

```
Abs(int a, double b)
```

```
    friend ostream &operator<<(ostream &out, const Abs &a){
```

```
        out << a.value_a << ':' << a.value_b;
```

```
        return out;
```

```
    }
```

```
    virtual void print() = 0;
```

```
};
```

```
class Class : private Abs{
```

```
public:
```

```
    friend ostream &operator<<(ostream &out, const Class &c);
```

```
};
```

```
friend ostream& operator<<(ostream& out, const Class& c) {  
    out << static_cast<const Abs&>(c); // Вызываем оператор для родительского класса  
    return out;  
}
```

2. Что такое поток? Какие типы потоков существуют? Какие библиотеки требуется подключить при использовании стандартных, файловых и строковых потоков? Сформулировать алгоритм удаления записей из файла.

Поток - ввод и вывод данных. Механизм для обмена данными между программой и внешними источниками.

1. Стандартные потоки: `#include <iostream>`

2. Файловые потоки: `#include <fstream>`

3. Строковые потоки: `#include <sstream>`

Алгоритм удаления записей из файла:

1. Открыть файл в режиме чтения и записи (`std::fstream`).
2. Создать временный файл для копирования данных.
3. Читать запись из исходного файла.
4. Если запись не нужно удалять, записать ее во временный файл.
5. После завершения чтения и записи, удалить исходный файл.
6. Переименовать временный файл в имя исходного файла.

Экзаменационный билет №16

1. Объясните какой конструктор был использован в коде ниже, а также измените класс (и main функцию) так, чтобы продемонстрировать вызов обоих конструкторов.

```
class CPP_Quest{
public:
    CPP_Quest() : somedata(nullptr) {}
    CPP_Quest(void *somedata = nullptr) : somedata(somedata) {}
    void *somedata;
};

int main(){
    CPP_Quest q();
    return 0;
}

CPP_Quest(void* somedata) : somedata(somedata)
```

```
CPP_Quest q1; // Вызов конструктора по умолчанию
CPP_Quest q2(data); // Вызов конструктора с параметром
```

2. Что такое поток? Какие типы потоков существуют? Какие библиотеки требуется подключить при использовании стандартных, файловых и строковых потоков? Сформулировать алгоритм добавления записей в файл.

Поток - ввод и вывод данных. Механизм для обмена данными между программой и внешними источниками.

1. Стандартные потоки: `#include <iostream>`
2. Файловые потоки: `#include <fstream>`
3. Строковые потоки: `#include <sstream>`
 1. Открыть файл в режиме записи (`ofstream`).
 2. Проверить, был ли файл успешно открыт.
 3. Если файл открыт успешно, записать новую запись в файл используя оператор записи (`<<` для `ofstream`).
 4. После завершения записи, закрыть файл.

Экзаменационный билет №17

1. В процессе изучения шаблонов в C++ Аркадий наткнулся на плохую и документацию и теперь его код не компилируется. Покажите Аркадию, как правильно пользоваться шаблонами в классах в C++.

```
template <typename T>
class<T>{
    T a;
    T b;

public:
    AB<T>(T a, T b) : a<T>(a), b<T>(b) {}
    T *func<T>(T c){
        a += c;
        b -= c;
        return a * b;
    }
};
```

```
template <typename T>
class AB {
    T a;
    T b;

public:
    AB(T a, T b) : a(a), b(b) {}

    T func(T c) {
        a += c;
        b -= c;
        return a * b;
    }
};
```

2. Из каких частей состоит библиотека STL? Какие типы контейнеров в ней существуют? Что представляет собой итератор? Какие операции можно выполнять над итераторами? Привести фрагмент кода на C++ с функцией для печати контейнера vector с использованием итератора.

Библиотека STL состоит из контейнеров, алгоритмов и итераторов

Контейнеры: ассоциативные (set), последовательностей(list), адаптеры(stack)

Итератор - это объект, который позволяет перемещаться по элементам контейнера, обеспечивая доступ к элементам.

Операции: инкремент, декремент, разыменовывание, сравнение

```
void printVector(const vector<T>& vec) {
    for (it = vec.begin(); it != vec.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;
}
```

Экзаменационный билет №18

1. "Обработка ошибок – это удобно", так подумал Артур, пока не столкнулся со строковыми ошибками. Помогите Артуру исправить код.

```
void ErrFunc() {  
    throw "Как тебя поймать? Как тебя поймать?!";  
}
```

```
void ErrHandlerFunc() {  
    try {  
        ErrFunc();  
    } (Ваш код) {  
        std::cout << "Ошибка!" << std::endl;  
    }  
}
```

```
void ErrFunc() {  
    throw runtime_error("Как тебя поймать? Как тебя поймать?!");  
}  
  
void ErrHandlerFunc() {  
    try {  
        ErrFunc();  
    }  
    catch (const exception& e) {  
        cout << "Ошибка! " << e.what() << endl;  
    }  
}
```

2. Из каких частей состоит библиотека STL? Какие типы контейнеров в ней существуют? Что представляют собой адаптеры контейнеров? Перечислить операции и методы общие для всех контейнеров и привести примеры использования некоторых из них.

Библиотека STL состоит из контейнеров, алгоритмов и итераторов

Контейнеры: ассоциативные (set), последовательностей(list), адаптеры(stack)

Методы: insert(), erase(), size(), empty(), find(), begin(), end()

```
vector<int> vec = { 4, 2, 5, 1, 3 };  
sort(vec.begin(), vec.end());
```

Экзаменационный билет №19

1. Реализуйте функцию добавления элемента в стек в середину.

```
class StackElement{
public:
    int value;
    StackElement(int value) : value(value) {}
};
```

```
class Stack{
private:
    StackElement *head;
    size_t size = 0;
};
```

```
void push(int value) {
    StackElement* newElement = new StackElement(value);
    newElement->next = head;
    head = newElement;
    size++;
}
```

2. Что представляет собой ассоциативный контейнер? Какие ассоциативные контейнеры существуют в библиотеке STL? Привести примеры методов, используемых в ассоциативных контейнерах, а также примеры использования некоторых из них.

Ассоциативные контейнеры - это контейнеры, которые хранят элементы в отсортированном порядке и обеспечивают быстрый поиск по ключу.

Существуют set, map, multiset, multimap

Методы: erase(), insert(), size(), empty(), find()

```
map<int, string> myMap;
myMap.insert({ 1, "One" });
```

Экзаменационный билет №20

1. Объясните, что будет выведено в результате вызова A().func() в функции main().

```
class A{
    void func(){
        cout << '1' << endl;
        class A{
        public:
            void func(){
                cout << '2' << endl;
                class A : public ::A{
                public:
                    void func(){
                        cout << '3' << endl;
                    }
                };
                ::A a = A();
                a.func();
            }
        };
        A a = A();
        a.func();
        cout << '4' << endl;
    }
};
```

Из-за вложенности классов и создания объектов, в конечном итоге вызывается `func()` третьего класса `A`, который выводит `3`.

2. Что представляет собой ассоциативный контейнер? Какие ассоциативные контейнеры существуют в библиотеке STL? Каким образом можно задать контейнер map? Привести для этого пример кода на C++.

Ассоциативные контейнеры - это контейнеры, которые хранят элементы в отсортированном порядке и обеспечивают быстрый поиск по ключу.

Существуют set, map, multiset, multimap

```
map<int, string> myMap;
```

Экзаменационный билет №21

1. Программист Евгений моделирует дорожную сеть, его интересуют несколько видов транспорта:

1. Легковая машина;
2. Грузовик;
3. Мотоцикл;
4. Самокат;

Помимо того, что все они нужны для передвижения, они имеют свои специальные функции.

Александру необходимо не только реализовать их общий класс и произвести наследование, но и реализовать специальные методы, чтобы:

1. Грузовик – перевозил грузы;
2. Легковая машина – перевозила людей;
3. Мотоцикл – передвигался с большой скоростью;
4. Самокат – передвигался в пешеходных зонах;

Помогите Александру реализовать данную задачу. В качестве работы специальных методов используйте печать на экран через `std::cout`.

```
class Transport {
public:
    virtual void move() const;
};

class Car : public Transport {
public:
    void move() const override;
};

class Truck: public Transport {
public:
    void move() const override;
};

ИТД
```

2. Для чего используется конструктор с параметрами? Пустой конструктор? Конструктор копирования?

Привести фрагмент кода на C++ с описанием класса и описанием каждого из конструкторов.

Конструктор с параметрами используется для инициализации объекта класса с заданными значениями при его создании.

Конструктор без параметров используется для создания объекта без передачи аргументов.

Конструктор копирования копирует значения существующего объекта класса в новый объект.

```
class Person {
public:
    // Конструктор с параметрами
    Person(string N, int A) {
        name = N;
        age = A;
    }

    // Конструктор без параметров
    Person() {
        name = "";
        age = 0;
    }

    // Конструктор копирования
    Person(const Person& P) {
        name = P.name;
        age = P.age;
    }
private:
    string name;
    int age;
};
```

Экзаменационный билет №22

1. Полгода назад Валерий создавал игру. В игре присутствуют различные монстры со своими шкалами здоровья, выносливости и магии. Валерий решил сделать рефакторинг старого кода. Помогите Валерию сделать это. Вот его код:

```
class Monstr_Boss_1 {
    int h = 10;
    int s = 2;
    int m = 3;
}
class Monstr_Boss_2 {
    int h = 20;
    int s = 3;
    int m = 5;
}
class Monstr_Boss_3 {
    int h = 40;
    int s = 60;
    int m = 10;
}
int main() {
    Monstr_Boss_1 monstr_boss_1;
    Monstr_Boss_2 monstr_boss_2;
    Monstr_Boss_3 monstr_boss_3;
}

class Monster {
protected:
    int health;
    int stamina;
    int magic;

public:
    Monstr(int h, int s, int m);
    int getHealth() const { return health; }
    int getStamina() const { return stamina; }
    int getMagic() const { return magic; }
};
```

2. Для чего используются дружественные функции и классы? Чем отличается перегрузка префиксных и постфиксных унарных операций? Каким образом можно перегрузить операции ввода-вывода? Привести фрагмент кода на C++ с описанием класса и определением перегруженных операций ввода-вывода.

Дружественные функции и классы в C++ предоставляют доступ другим функциям или классам к закрытой части (приватной или защищенной) другого класса.

Префиксный ++ и – выполняется до возвращения значения

Постфиксный ++ и – возвращает значение и выполняет операцию

Для перегрузки операторов ввода-вывода необходимо перегрузить operator<< operator>>

```
class Person {
public:
    void setAge(int A) { age = A; }
    int getAge() { return age; }
    friend ostream& operator <<(ostream& out, const Person& P);
    friend istream& operator >>(istream& in, Person& P);

private:
    int age;
};
```


Экзаменационный билет №23

1. Неопытный программист Слава решил разобраться в принципе множественного наследования. Слава хочет вызвать метод класса A, но программа не компилируется. Объясните, почему программа не работает, а также предложите два варианта решения данной проблемы. Вот его код:

```
class A {
    int a;
public:
    void method() { cout << "method A" << endl; }
};
class B {
    int b;
public:
    void method() { cout << "method B" << endl; }
};
class C : public B, public A {
    int c;
public:
    void foo() {
        method();
    }
};
int main() {
    C c;
    c.foo();
}
```

1. Разрешение области видимости

```
class C : public B, public A {
    int c;
public:
    void foo() {
        A::method(); // Вызов метода method из класса A
    }
};
```

2. Виртуальные функции

```
virtual void method() { cout << "method A" << endl; }
```

2. Что такое шаблон? Перечислите основные свойства шаблонов функций. Приведите пример неявного инстанцирования шаблона.

Шаблон – обобщение класса или функции, работающий с разными типами данных во избежание дублирования функции/класса.

Свойства:

- Определяется 1 раз
- Определение шаблона параметризуется
- Не могут быть виртуальными
- Могут быть производными от шаблона или от класса
- Могут являться базовыми

Инстанцирование – использование шаблонного класса с конкретным типом данных

```
Array<int> intArray(5);
```

Экзаменационный билет №24

1. Имеется иерархия классов:

```
class Student{
    int age;
public:
    string name;
};
class Employee: public Student{
protected:
    string post;
};
class Teacher: public Employee{
protected:
    int stage;
};
```

Для классов Student, Employee и Teacher написать конструкторы без параметров, конструкторы с параметрами, конструкторы копирования, а также определить операцию присваивания.

```
Student() {
    age = 0;
    name = "";
}

Student(int A, const string& N){
    age = A;
    name = N;
}

Student(const Student& S) {
    name = S.name;
    age = S.age;
}
```

2. Для чего используется конструктор с параметрами? Пустой конструктор? Конструктор копирования? Привести фрагмент кода на C++ с описанием класса и описанием каждого из конструкторов.

Конструктор с параметрами используется для инициализации объекта класса с заданными значениями при его создании.

Конструктор без параметров используется для создания объекта без передачи аргументов.

Конструктор копирования копирует значения существующего объекта класса в новый объект.

```
class Person {
public:
    // Конструктор с параметрами
    Person(string N, int A) {
        name = N;
        age = A;
    }

    // Конструктор без параметров
    Person() {
        name = "";
        age = 0;
    }

    // Конструктор копирования
    Person(const Person& P) {
        name = P.name;
        age = P.age;
    }
private:
    string name;
    int age;
};
```

Экзаменационный билет №25

1. Задан контейнер vector и его итератор:

```
class Iterator{  
    ...  
private:  
    int*elem;  
}  
class Vector{  
    ...  
private:  
    int size;  
    int*data;  
};
```

Как удалить из vector элементы со 2 по 5?

```
Iterator it;  
it.elem = myVector.data + 1; // Установка итератора на второй элемент (нумерация с 0)  
for (int i = 0; i < 4; ++i) {  
    myVector.data[i + 1] = *it.elem;  
    ++it.elem;  
}
```

2. Что представляет собой ассоциативный контейнер? Какие ассоциативные контейнеры существуют в библиотеке STL? Каким образом можно задать контейнер map? Привести для этого пример кода на C++.

Ассоциативные контейнеры - это контейнеры, которые хранят элементы в отсортированном порядке и обеспечивают быстрый поиск по ключу.

Существуют set, map, multiset, multimap

```
map<int, string> myMap;
```

Экзаменационный билет №26

1. Задан контейнер stack:

```
class Stack{  
    ...  
private:  
    int size;  
    int*data;  
};
```

Как удалить из него элемент с заданным номером? Какой контейнер удобнее для выполнения данной задачи: список или stack? Почему?

```
void delete(int index) {  
    if (index < 0 || index >= size) {  
        return;  
    }  
  
    for (int i = index; i < size - 1; i++) {  
        data[i] = data[i + 1];  
    }  
  
    size--;
```

Список удобнее, т.к. в стеке принцип LIFO – понадобится больше времени, чтобы найти элемент, удалить, восстановить стек.

2. Что такое полиморфные функции? Чем полиморфизм отличается от принципа подстановки? В каких случаях используется механизм позднего связывания? Привести фрагмент кода на C++ с примером использования полиморфной функции.

Полиморфизм - способность объекта использовать методы аналогичным способом, независимо от типа этого объекта. Это реализуется через наследование и виртуальные функции в C++. Полиморфизм может быть достигнут с помощью методов-членов, а также с помощью перегрузки операторов.

Принцип подстановки - объекты определенного класса А могут заменяться их подклассами В без изменения корректности программы.

Используется, когда вызываемый метод определяется во время выполнения программы на основе типа объекта, к которому обращается метод.

```
void drawShape(const Shape& shape) {  
    shape.draw();  
}
```

Экзаменационный билет №27

1. Студент Михаил решил создать контейнер map и написать функцию для добавления элементов в контейнер map с помощью функции make_pair():

```
template <class T1, class T2>
void addElements(map<T1, T2>& m, int n) {
    T1 temp1 = n;
    T2 temp2;
    for (int i = 0; i < n; i += 2)
    {
        cin >> temp2;
        m.insert(make_pair(n++, temp2));
    }
}
```

Однако он допустил ошибки в данной функции. Найдите и исправьте ошибки, а также напишите, как можно было по-другому написать функцию для добавления элементов в контейнер map.

```
template <class T1, class T2>
void addElements(map<T1, T2>& m, int n) {
    for (int i = 0; i < n; i += 2) {
        T2 temp2;
        cin >> temp2;
        m.insert(make_pair(i, temp2));
    }
}
```

2. Что представляет собой исключение на C++? На какие части исключения позволяют разделить вычислительный процесс? Каковы достоинства такого подхода?

Исключение - непредвиденное аварийное событие. В C++ - объект, который система должна генерировать при возникновении исключительной ситуации.

- Обнаружение аварийной ситуации (неизвестно как обрабатывать)
- Обработка аварийной ситуации (неизвестно где возникла)
- Удобно в программе из нескольких модулей.
- Не требуется возвращать значение в вызывающую функцию

Экзаменационный билет №28

1. Студенту Данилу задали привести примеры создания контейнера set. Он привёл 2 примера:

Создание пустого контейнера с помощью конструктора по умолчанию:

```
set<int> mySet;
```

Создание пустого контейнера с заданным компаратором:

```
struct Compare {  
    bool operator()(int a, int b) const {  
        return a > b;  
    }  
};  
set<int, Compare> mySet(Compare());
```

Приведите ещё примеры создания контейнера set.

```
vector<int> vec = { 1, 3, 5, 2, 4 };  
set<int> mySet(vec.begin(), vec.end());
```

```
set<int> mySet = { 4, 1, 3, 2, 5 };
```

2. Для чего используется принцип наследования? Что представляют собой виртуальные функции и механизм позднего связывания? Могут ли быть виртуальными конструкторы? Деструкторы? В чём заключается принцип подстановки? Привести фрагмент кода на C++, показывающий работу принципа подстановки.

Наследование – описание нового класса (дочернего) на основе существующего (родительского). Виртуальные функции позволяют определить функцию в родительском классе и переопределить в дочернем.

Полиморфизм (механизм позднего связывания) – вызывает методы дочернего класса через родительский.

Конструкторы и деструкторы могут быть виртуальными, но они не наследуются, поэтому смысла нет.

Принцип подстановки – объекты одного типа могут быть заменены объектами другого типа.

1. Молодой программист написал код, иллюстрирующий работу шаблонов класса, но допустил ряд ошибок:

```
//Point.h
#ifndef POINT_H
#define POINT_H
template<class T>
class Point{
    T x,y; //координаты точки
public:
    Point(int X=0,int Y=0):x(X),y(Y){}
    void Show () const;
};
#endif

//main.cpp
#include "Point.h"
template<class T>
void Point<T>::Show(){
    cout<<"("<<x<<" , "<<y<<")";
}
template<class T>
void main(){
    Point<int>p1;
    Point<T>p2(1,1);
    p1.Show();
    p2.Show();
}
```

Найдите и исправьте эти ошибки, а также расскажите про шаблоны, опираясь на данный код.

Шаблоны классов позволяют создавать обобщенные классы или функции, которые могут работать с различными типами данных без изменения их кода.

2. Что такое поток? Какие типы потоков существуют? Какие библиотеки требуется подключить при использовании стандартных, файловых и строковых потоков? Сформулировать алгоритм удаления записей из файла.

Поток - ввод и вывод данных. Механизм для обмена данными между программой и внешними источниками.

1. Стандартные потоки: `#include <iostream>`
2. Файловые потоки: `#include <fstream>`
3. Строковые потоки: `#include <sstream>`

Алгоритм удаления записей из файла:

1. Открыть файл в режиме чтения и записи (`std::fstream`).
2. Создать временный файл для копирования данных.
3. Читать запись из исходного файла.
4. Если запись не нужно удалять, записать ее во временный файл.
5. После завершения чтения и записи, удалить исходный файл.
6. Переименовать временный файл в имя исходного файла.

Экзаменационный билет №30

1. Молодому программисту Егору поставили задачу представить множество сотрудников компании и посетителей. Для этого он решил написать 2 класса:

```
class Person{
public:
    Person(const string& name, unsigned age) : name(name), age(age) {}
    void print() const{
        cout << "Name: " << name << "\tAge: " << age << endl;
    }
protected:
    string name;    // имя
    unsigned age;   // возраст
};
class Employee : public Person {
public:
    Employee(const string& name, unsigned age, const string& company) : Person(name, age),
    company(company) {}
    void print() const{
        Person::print();
        cout << "Name: " << name << "\tAge: " << age << endl;
    }
private:
    string company; // компания
};
```

Однако более опытный программист Кирилл сразу же “забраковал” работу. Исправьте данный код.

2. Для чего используется конструктор с параметрами? Пустой конструктор? Конструктор копирования? Привести фрагмент кода на C++ с описанием класса и описанием каждого из конструкторов.

Конструктор с параметрами используется для инициализации объекта класса с заданными значениями при его создании.

Конструктор без параметров используется для создания объекта без передачи аргументов.

Конструктор копирования копирует значения существующего объекта класса в новый объект.

```
class Person {
public:
    // Конструктор с параметрами
    Person(string N, int A) {
        name = N;
        age = A;
    }

    // Конструктор без параметров
    Person() {
        name = "";
        age = 0;
    }

    // Конструктор копирования
    Person(const Person& P) {
        name = P.name;
        age = P.age;
    }
private:
    string name;
    int age;
};
```